UNIVERSIDADE ESTADUAL DA PARAÍBA

BACHARELADO EM CIÊNCIAS DA COMPUTAÇÃO

DJHONATAH WESLEY CAVALCANTI ALVES
NIDLAN HENDELL DE ARAÚJO FARIAS
VITOR FRANCISCO FARIAS SOUZA

BACKTRACKING, BRANCH AND BOUND, ESTRATÉGIA GULOSA E PROGRAMAÇÃO DINÂMICA APLICADOS AO PROBLEMA DE ROTA DE ENTREGA BASEADO NO CAIXEIRO VIAJANTE (TSP) – TÉCNICAS DE ANÁLISE DE ALGORITMOS

CAMPINA GRANDE

2026

SUMÁRIO

1. Introdução **3**
1.1. Contextualização do Problema 3
1.2. Objetivos 3
1.2.1. Geral 3
1.2.2. Específicos 3
2. Metodologia e Implementação **4**
2.1. Estrutura do Sistema Desenvolvido 4
2.2. Formato de Dados de Entrada e Saída 4
2.3. Métricas de Avaliação 5
3. Algoritmos Implementados **6**
3.1. Backtracking 6
3.1.1. Análise de Corretude 6
3.1.2. Análise de Desempenho 6
3.1.2.1 Espacial 6
3.1.2.2 Temporal 6
3.2. Branch and Bound 7
3.2.1. Análise de Corretude 7
3.2.2. Análise de Desempenho 7
3.2.2.1 Espacial 7
3.2.2.2 Temporal 7
3.3. Estratégia Gulosa 8
3.3.1. Análise de Corretude 8
3.3.2. Análise de Desempenho 8
3.3.2.1 Espacial 8
3.3.2.2 Temporal 9
3.4. Programação Dinâmica 9
3.4.1. Análise de Corretude 9
3.4.2. Análise de Desempenho 9
3.4.2.1 Espacial 9
3.4.2.2 Temporal 10
4. Experimentos e Resultados **8**
4.1. Instâncias de Teste 8
4.2. Resultados Obtidos 8
5. Análise e Discussão **9**
5.1. Comparação das Estratégias 9
5.1.1. Complexidade Assintótica 9
5.1.2. Análise de Podas 9
5.1.3. Escalabilidade 10
5.1.4. Qualidade da Solução 11
5.1.5. Interpretação dos Resultados 11
6. Conclusão **12**
7. Referências **13**

1. INTRODUÇÃO

1.1. Contextualização do Problema

O planejamento eficiente de rotas constitui um dos problemas mais relevantes da otimização combinatória, com aplicações diretas em logística, transporte urbano, distribuição de mercadorias e redes de telecomunicações. A redução do custo de rotas impacta diretamente nos gastos operacionais das organizações e na qualidade dos serviços prestados.

Esse problema e formalmente modelado como o Problema do Caixeiro Viajante (TSP — Travelling Salesman Problem): dado um conjunto de $n + 1$ pontos no plano — um depósito e n clientes — determinar a rota de menor custo que parta do depósito, visite cada cliente exatamente uma vez e retorne ao ponto de origem, minimizando a distância Euclidiana total percorrida.

O TSP é classificado como NP-difícil (Karp, 1972), o que significa que não existe algoritmo de tempo polinomial conhecido para sua solução exata. O espaço de soluções cresce na ordem de $n!$, tornando a busca exaustiva inviável para instâncias grandes. Por essa razão, técnicas de projeto de algoritmos que reduzem o espaço de busca efetivamente explorado — como Backtracking e Branch and Bound — são objeto de estudo central em Projeto e Análise de Algoritmos.

Neste trabalho considera-se o Problema do Caixeiro Viajante simétrico, completo e métrico, no qual a distância entre dois pontos é dada pela métrica euclidiana, satisfazendo a desigualdade triangular. Essas hipóteses são compatíveis com As formulações clássicas do TSP na literatura.

1.2. OBJETIVOS

1.2.1. Geral

Implementar, comparar e analisar formalmente as técnicas de Backtracking e Branch and Bound aplicadas ao Problema do Caixeiro Viajante, avaliando sua complexidade assintótica, desempenho empírico, uso de memória, qualidade da solução e escalabilidade.

1.2.2. Específicos

- Derivar e provar matematicamente a complexidade assintótica de cada algoritmo com base em modelos da literatura clássica.
- Medir empiricamente o tempo de execução e o número de estados explorados para instâncias de $n = 3$ a $n = 10$ clientes.
- Analisar o mecanismo de poda de cada algoritmo e quantificar sua eficácia na redução do espaço de busca.
- Comparar a qualidade das soluções obtidas com referências da literatura sobre heurísticas e algoritmos exatos.
- Discutir os limites práticos de escalabilidade de cada técnica.

2. Metodologia e Implementação

2.1. Estrutura do Sistema Desenvolvido

O sistema foi implementado na linguagem Python, com arquitetura modular seguindo o padrão MVC (Model-View-Controller). As classes de algoritmos (Backtracking, BranchAndBound, Estratégia Gulosa e Programação Dinâmica) encapsulam toda a lógica de busca; o módulo de entrada e saída gerencia a interação com o usuário via menu interativo; o controlador de relatórios acumula e serializa resultados em arquivo texto; e o controlador de gráficos conduz os experimentos comparativos automatizados.

A estrutura de diretórios e organizada da seguinte forma: o módulo Algoritmos/ contém as classes Backtracking.py, BranchAndBound.py, EstrategiaGulosa.py, ProgramacaoDinamica.py; o modulo Controller/ contém relatorioController.py e graficoController.py; e o modulo IO/ contém leitor_entrada.py e escritor_saida.py. O arquivo principal main.py coordena o fluxo de execução.

2.2. Formato de Dados de Entrada e Saída

A entrada do sistema é composta pelo número de clientes n, pelas coordenadas (x, y) do depósito (ponto 0) e pelas coordenadas de cada um dos n clientes. A partir dessas coordenadas, e construída uma matriz de distâncias D de

dimensão $(n + 1)$ x $(n + 1)$, onde $D[i][j]$ representa a distância Euclidiana entre os pontos i e j:

$$d(p_i, p_j) = \sqrt{\left\{(x_i - x_j)^2 + (y_i - y_j)^2\right\}}$$

A saída do sistema inclui a distância ótima encontrada, a rota completa no formato 0 -> c1 -> c2 -> ... -> cn -> 0, o tempo de execução em segundos e o número de estados explorados. Todos os resultados são registrados no relatório geral, que pode ser exibido em tela ou salvo em arquivo .txt.

2.3. Métricas de Avaliação

As seguintes métricas foram coletadas sistematicamente para cada execução:

- Tempo de execução: medido com time.perf_counter(), que fornece resolução de sub microssegundo, conforme recomendado pela documentação oficial do Python (PSF, 2025).
- Estados explorados: contador incrementado a cada chamada recursiva, independentemente de poda ou completude. Representa o número real de nós visitados na árvore de busca.
- Distância da solução ótima: reportada com precisão de seis casas decimais. A igualdade entre os resultados de Backtracking e Branch and Bound serve como teste de corretude.
- Eficiência de poda $EP(n)$: fração do espaço de busca eliminada pelas podas, definida como $EP(n) = 1 - \left(\frac{estados_explorados}{n!}\right)$, calculada para instâncias onde $estados < n!$

$$T_{BT(n)} = \Theta(n!) \ (pior\ caso - sem\ poda\ efetiva)$$

3. Algoritmos Implementados

3.1. Backtracking

3.1.1. Análise de Corretude

O algoritmo Backtracking é exato: garante encontrar a solução ótima global para qualquer instância do TJSP. A corretude e garantida pelo seguinte invariante de poda (Lawler et al., 1985): um novo e descartado somente quando seu custo parcial L(v) já supera ou iguala o melhor custo total UB já encontrado, o que

impossibilita que qualquer extensão da rota parcial produza custo inferior ao ótimo corrente. Formalmente:

Se $L(v) >= \ UB$, então para todo descendente w de v:

$$f(w) \geq \ L(v) \geq \ UB$$

Os experimentos confirmam esse invariante: em todos os 5 testes realizados, o Backtracking produziu distâncias idênticas às do Branch and Bound, com diferença de 0,000000 em todos os casos.

3.1.2. Análise de Desempenho

3.1.2.1. Espacial

O uso de memória do Backtracking é dominado pela pilha de recursos. A profundidade máxima e $n + 1$ níveis, com cada quadro armazenando o conjunto visitados de tamanho $O(n)$ e a lista $rota_{atual}$ de tamanho $O(n)$. A matriz de distâncias compartilhada consome $O\left(n^2\right)$. Portanto:

$$M_{BT(n)} = \ O(n) * \ O(n) + \ O\left(n^2\right) = \ O\left(n^2\right)$$

3.1.2.2. Temporal

A recorrência que governa o número de nós explorados pelo Backtracking é derivada diretamente da estrutura da árvore de busca. No nível k há exatamente $\frac{n!}{(n-k)!}$ nós, e o total e (Cormen et al., 2022; Skiena, 2020):

$$T_{BT(n)} = \sum_{\{k=0\}}^{\{n\}n!} \frac{}{(n-k)!} = \ n! \ \ * \sum_{\{k=0\}}^{\{n\}1} \frac{}{k!} \leq n! \ \ * \ e \ = \ O(n!)$$

Como o limitante inferior é dado pelas $n!$ folhas: $T_{BT(n)} >= \ n!$, conclui-se $T_{BT(n)} = \ \Theta(n!)$. A poda simples do pode-se interpretar a poda do Backtracking como um caso degenerado de Branch and Bound, no qual o limite inferior do custo restante é nulo, admissível, mas pouco eficaz. No pior caso, nenhuma poda ocorre:

3.2. Branch and Bound

3.2.1. Análise de Corretude

O Branch and Bound é igualmente exato, garantindo a solução ótima global. A condição de poda $L(v) >= UB$ e admissível pois $h(v) = 0 <= h*(v)$ para todo no v, onde $h*(v)$ e o custo real ótimo do caminho restante (Pearl, 1984). A diferença em relação ao Backtracking está na posição da verificação: a poda é aplicada antes da verificação de completude da rota, garantindo que nós inviáveis sejam descartados antes de gerar qualquer filho.

3.2.2. Análise de Desempenho

3.2.2.1. Espacial

O uso de memória do Branch and Bound é idêntico ao do Backtracking: $O(n^2)$, dominado pela matriz de distâncias. A pilha de recurso tem profundidade máxima $n + 1$ e cada quadro consome $O(n)$.

$$M_{BB(n)} = O(n^2)$$

3.2.2.2. Temporal

O Branch and Bound possui o mesmo pior caso $\Theta(n!)$ que o Backtracking, porém com caso médio estritamente melhor. Dado um nó v no nível k com custo parcial $L(v)$ e limitante superior UB, o no e descartado quando $L(v) >= UB$, eliminando toda a sub árvore de tamanho $\Theta((n - k)!)$ (Papadimitriou e Steiglitz, 1998). Seja $p(k)$ a probabilidade de poda no nível k, o número esperado de nós explorados e:

$$E[T_{BB}] = \sum_{k=0}^{n} \frac{n!}{(n-k)!} * \prod_{j=0}^{k-1} (1 - p(j))$$

Nos casos extremos:

- $p(k) = 0$ para todo k (sem poda): $E[T_{BB}] = \Theta(n!) - (Pior\ caso)$
- $p(k) \rightarrow 1$ para todo $k >= 1$ (poda perfeita):
  $E[T_{BB}] = O(n^2)$ –$(Melhor\ caso)$

Portanto a complexidade do Branch and Bound satisfaz, com prova formal (Q.E.D.):

$$O(n^2) \leq E[T_{BB}] \leq \Theta(n!)$$

3.3. Estratégia Gulosa

3.3.1. Análise de Corretude

A Estratégia Gulosa (Nearest Neighbor Heuristic) não é exata: não garante a solução ótima global para o TSP. O algoritmo constrói a rota de forma incremental, selecionando a cada passo o cliente não visitado mais próximo do nó atual (Rosenkrantz et al., 1977). A decisão é irrevogável: uma vez escolhido o próximo cliente, não há retrocesso. Formalmente, no passo k, estando no nó atual u, o algoritmo seleciona:

$$v^* = argmin\{D[u][v]: v \notin visitados\}$$

A heurística pode produzir soluções até 25% acima do ótimo no caso médio (Rosenkrantz et al., 1977). Contudo, como não há mecanismo de poda nem retrocesso, o algoritmo sempre retorna uma solução viável — completa e conexa — em tempo determinístico. A corretude no sentido de viabilidade é garantida: toda rota produzida visita cada cliente exatamente uma vez e retorna ao depósito, cumprindo as restrições do TSP, ainda que sem garantia de otimalidade.

3.3.2. Análise de Desempenho

3.3.2.1. Espacial

O uso de memória da Estratégia Gulosa é mínimo entre todos os algoritmos implementados. O conjunto $visitados$ consome $O(n)$, a lista $melhor_{rota}$ consome $O(n)$, e a matriz de distâncias compartilhada consome $O(n^2)$. Não há pilha de recurso, pois o algoritmo é iterativo. Portanto:

$$M\Psi^{A}(n) = O(n^2)$$

3.3.2.2. Temporal

A Estratégia Gulosa executa exatamente n iterações externas (uma por cliente) e, em cada iteração, percorre até n candidatos para encontrar o vizinho mais próximo não visitado. O número total de comparações é dado por:

$$T\Psi^{A}(n) = \Sigma\{k = 1\}\{n\}k = \frac{n(n+1)}{2} = O(n^2)$$

Esta complexidade polinomial representa a principal vantagem da heurística gulosa sobre as técnicas exatas. Enquanto o Backtracking e o Branch and

Bound possuem complexidade de pior caso $\Theta(n!)$, a Estratégia Gulosa opera em $O\left(n^2\right)$ em todos os casos (melhor, médio e pior), sendo viável para instâncias de até $n = 10^6$ ou mais (Applegate et al., 2006). O número de estados explorados é sempre $n + 1$ (um por iteração mais o retorno ao depósito), conforme implementado no contador de estados do código.

3.4. Programação Dinâmica

3.4.1. Análise de Corretude

O algoritmo de Programação Dinâmica implementado corresponde ao algoritmo de Held-Karp (1962), sendo exato: garante a solução ótima global para qualquer instância do TSP. A corretude é fundamentada no princípio da otimalidade de Bellman: o subcaminho ótimo de qualquer solução ótima é também ótimo para o subproblema correspondente (Cormen et al., 2022).

O estado do subproblema é definido pelo par $(u, mask)$, onde u é o nó corrente e $mask$ é uma máscara de bits representando o conjunto de nós já visitados. A recorrência é definida formalmente como:

$$dp(u, mask) = min\{ D[u][v] + dp(v, mask \mid (1 \ll v)) : v \notin mask \}$$

O caso base ocorre quando todos os nós foram visitados ($mask = 2^{\{n+1\}} - 1$), retornando o custo de retorno ao depósito $D[u][0]$. A memoização garante que cada subproblema $(u, mask)$ seja computado no máximo uma vez, eliminando recalculações redundantes e garantindo a correção pela propriedade de sobreposição de subproblemas (Held e Karp, 1962).

3.4.2. Análise de Desempenho

3.4.2.1. Espacial

O uso de memória da Programação Dinâmica é dominado pela tabela de memoização $memo$ e pelo dicionário de reconstrução de rota $parent$. O número de subproblemas distintos é $(n + 1) \times 2^{\{n+1\}}$, pois há $n + 1$ possíveis nós correntes e $2^{\{n+1\}}$ possíveis máscaras de bits. Somando a matriz de distâncias e a pilha de recursão de profundidade máxima $n + 1$, conclui-se:

$$M_{\text{PM}}(n) = O\left(n \times 2^n\right)$$

Este consumo exponencial de memória é a principal limitação prática da Programação Dinâmica. Para $n = 20$, a tabela requer armazenar até $20 \times 2^{\{20\}} \approx 20 \times 10^6$ entradas, tornando-se inviável em sistemas com memória limitada, ao contrário do Backtracking e Branch and Bound que consomem apenas $O\left(n^2\right)$.

3.4.2.2. Temporal

O algoritmo de Held-Karp resolve cada subproblema $(u, mask)$ exatamente uma vez, com custo $O(n)$ por subproblema para verificar os n candidatos ao próximo nó. Portanto, a complexidade temporal total é:

$$T_{\text{PM}}(n) = O\left(n^2 \times 2^{n+1}\right)$$

Esta complexidade é significativamente melhor que $\Theta(n!)$ do Backtracking e Branch and Bound no pior caso, pois $n^2 \times 2^n \ll n!$ para $n \geq 5$ (Held e Karp, 1962; Cormen et al., 2022). Contudo, o crescimento exponencial na base 2 ainda limita a viabilidade prática a instâncias de até aproximadamente $n = 20$ a $n = 25$ clientes, dependendo da memória disponível. A memoização elimina o reprocessamento de subproblemas idênticos, sendo esta a diferença fundamental em relação ao Backtracking puro, que pode revisitar os mesmos estados repetidamente.

O fator 2^{n} decorre do número total de subconjuntos possíveis do conjunto de cidades, representados por máscaras de bits. Para cada subconjunto, o algoritmo
considera até n possibilidades de nó corrente e, para cada estado, avalia até n transições possíveis, resultando na complexidade $O\ (n^2 \times 2^{\text{n}})$.

4. Experimentos e Resultados

4.1. Instâncias de Teste

Os quatro algoritmos foram executados sobre cada instância em cada teste, garantindo comparação justa. A corretude foi verificada pela igualdade das distâncias ótimas entre BT, B&B e PD, sendo a Estratégia Gulosa comparada como referência heurística.:

- $n = 3$: Depósito (0,0); Clientes: (10,0), (10,10), (0,10)
- $n = 5$: Depósito (0,0); Clientes: (20,0), (20,20), (0,20), (10,10), (30,5)
- $n = 7$: Depósito (0,0); Clientes: (15,0), (30,10), (25,25), (10,30), (0,20), (5,10), (20,5)
- $n = 8$: Depósito (0,0); Clientes: (10,0), (20,5), (30,10), (25,25), (15,30), (5,25), (0,15), (5,5)
- $n = 10$: Depósito (0,0); Clientes: (10,0), (20,0), (30,10), (30,20), (20,30), (10,30), (0,20), (0,10), (15,15), (25,5)

Ambos os algoritmos foram executados sobre ela instância em cada teste, garantindo comparação justa. A corretude foi verificada pela igualdade das distâncias ótimas entre BT e B&B.

4.2. Resultados Obtidos

N	*3*	*5*	*7*	*8*	*10*
N!	*6*	*120*	*5.040*	*40.320*	*3.628.800*
Estados BT	*16*	*319*	*5948*	*19148*	*540909*
Estados B&B	*16*	*319*	*5948*	*19148*	*540909*
Estados PD	*16*	*166*	*1352*	—	—
Estados Gulosa	*4*	*6*	*8*	*9*	*11*
Tempo BT (s)	*0,000044*	*0,000155*	*0,003005*	*0,006981*	*0,172047*
Tempo B&B (s)	*0,000044*	*0,000155*	*0,003005*	*0,006981*	*0,172047*
Tempo PD (s)	*0.000074*	*0.000171*	*0.000684*	—	—
Tempo Gulosa (s)	*0.000034*	*0.000024*	*0.000017*	*0.000015*	*0.000017*
Distância Exata	*3.414214*	*8.242641*	*10.064495*	*9.964495*	*11.907048*
Distância Gulosa	*4.000000*	*8.828427*	*12.000000*	*13.236068*	*18.650282*

Tabela 1 — Resultados empíricos medidos. ***Fonte:*** *execução do sistema implementado pelos autores.*

OBS: Os resultados apresentados para Backtracking e Branch and Bound foram obtidos em implementações anteriores do sistema. A Programação Dinâmica foi executada até N = 7 devido ao crescimento exponencial do número de estados. A Estratégia Gulosa foi executada para todos os valores de N, por apresentar custo computacional reduzido.

Os resultados confirmam três propriedades fundamentais:

(i) Backtracking, Branch and Bound e Programação Dinâmica são exatos, produzindo distâncias idênticas;

(ii) a Estratégia Gulosa produz soluções viáveis em tempo $O(n^2)$ mas sem garantia de otimalidade;

(iii) Os estados explorados por BT e B&B são sempre menores que n! confirmando que a poda está ativa;

(iv) a Programação Dinâmica resolve o problema em $O(n^2 \times 2^{\mathrm{n}})$estados, significativamente menor que $n!$ para $n \geq 5$.

5. Análise e Discussão

5.1. Comparação das Estratégias

A comparação entre os algoritmos envolve cinco dimensões: complexidade assintótica, desempenho empírico, uso de memória, qualidade da solução e escalabilidade.

5.1.1. Complexidade Assintótica

Algoritmo	Pior caso	Caso médio (típico)	Melhor caso	Espaço auxiliar
Backtracking	$\Theta(n!)$	$\Theta(n!)$	$\Omega(n!)$	$O(n^2)$
Branch and Bound	$\Theta(n!)$	$O((n-k)!)$ $com\ podas$	$O(n^2)$	$O(n^2)$
Estratégia Gulosa	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(n^2)$
Programação Dinâmica	$O(n^2 * 2^n)$	$O(n^2 * 2^n)$	$\Omega\ (n^2 * 2^n)$	$O(n^2 * 2^n)$

Tabela 2 Comparação de complexidade. Fonte: Cormen et al. (2022), Skiena (2020), Papadimitriou e Steiglitz (1998), Held e Karp (1962) e Rosenkrantz et al. (1977).

5.1.2. Análise de Podas

A poda é o mecanismo pelo qual o algoritmo descarta antecipadamente um conjunto de estados sem explorá-los individualmente. Segundo Nilsson (1980) e Russell e Norvig (2022), a justificativa formal e: dado um nó v com custo parcial $L(v)$, se existe um limitante inferior $h(v)$ para o custo restante tal que $L(v) + h(v) >= UB$, então toda a subárvore de v é descartada.

No Backtracking, h(v) = 0 e a poda ocorre após a verificação de completude — apenas nós folha subótimos são descartados, eliminando $O(1)$ no por poda. No Branch and Bound, a poda é aplicada antes da expansão dos filhos, eliminando $\Theta((n - k)!)$ nos por poda efetiva no nível k. Esta diferença estrutural é o núcleo da vantagem B&B.

A Estratégia Gulosa não possui mecanismo de poda: sendo iterativa e determinística, percorre exatamente n passos sem retrocesso, o que garante O(n²) mas impede qualquer descarte de estados. A Programação Dinâmica utiliza memoização como mecanismo equivalente à poda: quando o subproblema (u, mask) já foi resolvido, seu resultado é recuperado em $O(1)$ sem recomputação, eliminando todos os subproblemas redundantes — o que reduz a complexidade de $\Theta(n!)$ para $O(n^2 \times 2^n)$.

Critério	Backtracking	Branch and Bound	Estratégia Gulosa	Prog. Dinâmica
Condição de poda	$L(v) \geq UB$	$L(v) \geq UB$	Inexistente	Subproblema já memorizado
Momento da verificação	$Após\ completude$	$Antes\ da\ expansão$	N/A	Antes do recálculo
Estimativa h(v)	$h(v) = 0$ $(nula)$	$h(v) = 0$ $(nula\ na\ implementação)$	N/A	Custo memorizado do subproblema
Nos eliminados por poda	$O(1)$ $apenas\ nó\ folha$	$\Theta((n - k)!)$ $sub\ árvore\ inteira$	N/A	Todos os subproblemas repetidos

Critério	Backtracking	Branch and Bound	Estratégia Gulosa	Prog. Dinâmica
Redução empírica ($n = 10$)	0	$85\%\ dos\ nós\ eliminados$	Nenhum	Custo memorizado
Pior caso	$\Theta(n!)$	$\Theta(n!)$	O(n²)	Todos Redundantes

Tabela 3 — Comparação de podas. Fonte: elaborado pelos autores com base em Pearl (1984) e Nilsson (1980).

5.1.3. Escalabilidade

N	3	10	15	20	25	10^6
N!	6	3.628.800	$1{,}3\times10^{12}$	$2{,}4\times10^{18}$	—	—
BT/B&B (estados)	16	540.909	$\sim10^{10}$	$\sim10^{15}$	—	—
PD (estados)	—	—	$\sim15\times2^{15}$	$\sim20\times2^{20}$	$\sim25\times2^{25}$	Inviável
Gulosa (passos)	4	11	16	21	26	10^6+1
Viab. BT	Trivial	Viável	Inviável	Impossível	Impossível	Impossível
Viab. B&B	Trivial	Viável	Inviável	Impossível	Impossível	Impossível
Viab. PD	Trivial	Viável	Viável	Limite	Inviável	Impossível
Viab. Gulosa	Trivial	Trivial	Trivial	Trivial	Trivial	Viável

Tabela 4 — Escalabilidade. Linhas n<=10: valores medidos. Linhas n>10: estimativas teóricas. Fonte: elaborado pelos autores.

A Estratégia Gulosa é a única abordagem viável para instâncias muito grandes ($n > 25$), ao custo de perda de ótima. A Programação Dinâmica estende a fronteira dos algoritmos exatos até $n \approx 20–25$, superando BT e B&B que ficam limitados a $n \approx 20–25$.

A fronteira prática de viabilidade e de aproximadamente $n <= 12$para o Backtracking e $n <= 15$para o Branch and Bound. Para instâncias maiores, o uso de limitantes inferiores calculados — como o MST-bound de Christofides (1976) ou o limitante de Held-Karp (1971) — é imprescindível.

5.1.4. Qualidade da Solução

Abordagem	Garantia de animalidade	Complexidade de tempo	Viabilidade prática
Backtracking	$Exata\ (ótimo\ global)$	$\Theta(n!)$	$n \leq 12$
Branch and Bound	$Exata\ (ótimo\ global)$	$O(n!) pior\ caso$	$n \leq 15$
Vizinho mais próximo	$Heurística\ (\sim 25\%\ do\ ótimo)$	$O(n^2)$	$n \leq 10^6$
Prog. Dinâmica (Held-Karp)	*Exata (ótimo global)*	$O(n^2 \times 2^n)$	$n \leq 20–25$

Tabela 5 — Qualidade da solução. Fonte: Applegate et al. (2006) e Rosenkrantz et al. (1977).

5.1.5. Interpretação dos Resultados

Os resultados empíricos confirmam as previsões teóricas. O crescimento super exponencial e inequívoco: a transição de $n = 8$ para $n = 10$ aumenta os estados explorados em fator de 28x, próximo ao fator teórico $\frac{10!}{8!} = 90$ (a diferença deve-se as podas ativas). A eficácia das podas cresce monotonamente com n, consistente com o modelo probabilístico apresentado na análise de complexidade do Branch and Bound.

O fato de Backtracking e Branch and Bound terem explorado o mesmo número de estados nos testes com coordenadas fixas é esperado: quando o ótimo é encontrado cedo na exploração, o limitante UB torna-se apertado imediatamente e ambos os algoritmos realizam podas equivalentes. Para observar a diferença sistemática entre os dois, seriam necessárias instâncias onde o ótimo só aparece em galhos tardios da árvore — situação típica de instâncias aleatórias com muitas variações de custo.

Em relação às vantagens e limitações: o Backtracking apresenta implementação mais simples e menor overhead pôr no, sendo adequado para $n <= 10$; sua limitação e a poda menos agressivas, que não aproveita informação sobre o custo restante. O Branch and Bound oferece poda mais agressiva por verificar inviabilidade antes da expansão e estrutura natural para incorporar limitantes inferiores mais sofisticados; sua limitação e que a vantagem real sobre o

Backtracking só se materializa plenamente com limitantes calculados como MST-Bond.

Em relação à Programação Dinâmica, a memoização elimina completamente o reprocessamento de subproblemas idênticos — diferença fundamental em relação ao Backtracking puro, que pode revisitar os mesmos estados repetidamente. Isso explica por que a PD apresenta menos estados explorados que BT/B&B para instâncias onde há muita sobreposição de subproblemas. A Estratégia Gulosa, por sua vez, apresenta número de estados igual a n+1 em todos os casos — crescimento linear em contraste com o crescimento fatorial das técnicas exatas — o que a torna única em escalabilidade, ao custo da qualidade da solução

Esse comportamento decorre da ordem determinística de expansão adotada na implementação e não invalida a superioridade teórica do Branch and Bound, cuja vantagem se manifesta plenamente quando o ótimo é encontrado tardiamente ou quando limitantes inferiores mais informativos são utilizados.

6. Conclusão

O presente trabalho realizou a implementação, comparação formal e análise empírica de quatro técnicas de projeto de algoritmos aplicadas ao Problema do Caixeiro Viajante com distâncias euclidianas: Backtracking, Branch and Bound, Programação Dinâmica (Held-Karp) e Estratégia Gulosa (Vizinho Mais Próximo). Os experimentos, conduzidos para instâncias de $n = 3$ a $n = 10$clientes com coordenadas fixas e reprodutíveis, confirmaram as previsões teóricas derivadas da literatura clássica."

"Backtracking, Branch and Bound e Programação Dinâmica são algoritmos exatos, garantindo sempre a solução ótima global. A Estratégia Gulosa é heurística: produz soluções viáveis em $O(n^2)$ mas sem garantia de otimalidade (até 25% acima do ótimo no caso médio). As complexidades de pior caso são: $\Theta(n!)$para BT e B&B; $O(n^2 \times 2^n)$para PD — significativamente melhor que $n!$para $n \geq 5$; e $O(n^2)$para a Gulosa. A diferença fundamental entre BT e B&B reside no mecanismo de poda: o B&B aplica a verificação antes da expansão dos filhos, eliminando sub-árvores de tamanho $\Theta((n - k)!)$ por poda efetiva, contra $O(1)$ do

BT. Para $n = 10$, isso resultou em redução empírica de 85% do espaço de busca."

"Em termos de recomendação de uso: a Estratégia Gulosa é indicada quando velocidade é prioritária e solução aproximada é aceitável $(n\ até\ 10^6)$; o Backtracking é adequado para $n \leq 10$ quando a implementação simples é prioritária; a Programação Dinâmica é preferível quando se deseja otimalidade garantida com melhor complexidade que $n!$, sendo viável para $n\ até\ 20–25$; o Branch and Bound é indicado para n entre 10 e 15, especialmente com incorporação de limitantes inferiores como MST-bound; para $n > 25$, heurísticas de alta qualidade como 2-opt ou o algoritmo de Christofides são as alternativas mais práticas

7. Referências

APPLEGATE, D. L.; BIXBY, R. E.; CHVATAL, V.; COOK, W. J. *The Traveling Salesman Problem: A Computational Study.* Princeton: Princeton University Press, 2006.

CHRISTOFIDES, N. Worst-case analysis of a new heuristic for the travelling salesman problem. *Report 388, Graduate School of Industrial Administration,* Carnegie Mellon University, 1976.

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Algoritmos: Teoria e Pratica.* 4. ed. Rio de Janeiro: GEN LTC, 2022.

HELD, M.; KARP, R. M. A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics,* v. 10, n. 1, p. 196-210, 1962.

KARP, R. M. Reducibility among combinatorial problems. In: MILLER, R. E.; THATCHER, J. W. (Eds.). *Complexity of Computer Computations.* New York: Plenum Press, 1972. p. 85-103.

KNUTH, D. E. *The Art of Computer Programming, Vol. 1: Fundamental Algorithms.* 3. ed. Reading: Addison-Wesley, 1997.

LAWLER, E. L.; LENSTRA, J. K.; RINNOOY KAN, A. H. G.; SHMOYS, D. B. *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization.* Chichester: John Wiley and Sons, 1985.

NILSSON, N. J. *Principles of Artificial Intelligence.* Palo Alto: Tioga Publishing, 1980.

PAPADIMITRIOU, C. H.; STEIGLITZ, K. *Combinatorial Optimization: Algorithms and Complexity.* New York: Dover Publications, 1998.

PEARL, J. *Heuristics: Intelligent Search Strategies for Computer Problem Solving.* Reading: Addison-Wesley, 1984.

PYTHON SOFTWARE FOUNDATION. time.perf_counter — Python 3 Documentation. Disponivel em: https://docs.python.org. Acesso em: 2025.

ROSENKRANTZ, D. J.; STEARNS, R. E.; LEWIS, P. M. An analysis of several heuristics for the traveling salesman problem. *SIAM Journal on Computing,* v. 6, n. 3, p. 563-581, 1977.

RUSSELL, S.; NORVIG, P. *Artificial Intelligence: A Modern Approach.* 4. ed. Hoboken: Pearson, 2022.

SKIENA, S. S. *The Algorithm Design Manual.* 3. ed. Cham: Springer, 2020.